



VIT[®]
B H O P A L
VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore
Madhya Pradesh - 466114

VITYARTHI PROJECT

REG.NO: 26BCE10367

NAME : SHASHWAT SHAURYA

BRANCH: COMPUTER SCIENCE (CORE)

PROJECT NAME: EXPENSE TRACKER

SEMESTER: Fall Semester 2026-27

CLASS REPORT

EXPENSE TRACKER

1. Introduction

The **Expense Tracker** is an interactive, modular command-line application built with Python to help users manage, organize and analyse personal expenses locally. The current implementation supports adding, viewing, filtering and deleting expense records, along with report generation and a budget alert.

Expense records contain an amount, category and date. The application validates user input before accepting a record and stores the data persistently in a structured JSON file. The program is divided into focused Python modules so that the main menu, expense operations, reporting, storage, validation and utility display logic remain separated.

The current repository README describes the project as a VITYarthi college project intended to use basic Python concepts while remaining readable and understandable to normal users. The repository also documents Python 3.8+, JSON storage, Git/GitHub and the command used to run the application. ■cite■turn0view0■

Project Scope

- Manage expense records through a console menu.
- Add new expenses after validating amount, category and date.
- View all stored expenses.
- Filter expenses by category.
- Delete an expense interactively.
- Generate total and category-wise spending reports.
- Identify the highest transaction.
- Warn the user when spending exceeds the ■10,000 budget threshold.
- Persist expense data using expenses.json.

2. Problem Statement

Maintaining day-to-day expenses manually can make it difficult to calculate total spending, compare categories and identify the largest transaction. A small, focused application can automate these repetitive calculations while keeping the underlying implementation understandable.

Problem Definition

Develop a Python-based Expense Tracker that accepts expense information, validates the input, stores records persistently, supports basic record management, filters records by category and generates useful spending summaries.

Core Record Structure

Field	Purpose
amount	Numeric expense amount; must be greater than zero.
category	Non-empty text category such as Food, Travel or Stationery.
date	Calendar date entered in YYYY-MM-DD format.

The GitHub repository currently describes the same three core data fields and the JSON persistence approach in its feature and project-structure documentation. ■ cite ■ turn0view0 ■

3. Functional Requirements

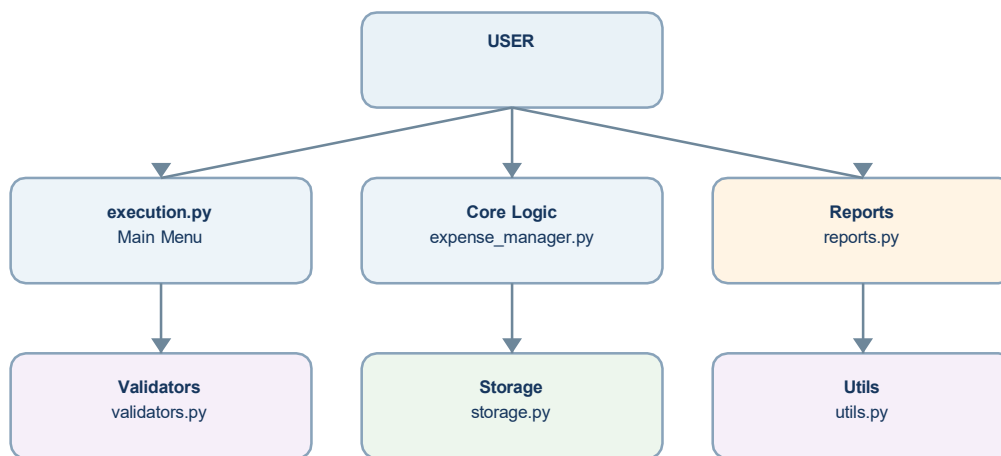
ID	Functional Requirement
FR-01	Add a new expense using amount, category and date.
FR-02	Reject non-positive expense amounts.
FR-03	Reject empty or invalid category input.
FR-04	Validate dates using YYYY-MM-DD parsing.
FR-05	Display all stored expense records.
FR-06	Filter expense records by category.
FR-07	Delete an expense using its displayed number.
FR-08	Calculate total expenses.
FR-09	Generate category-wise spending summaries.
FR-10	Identify the highest expense transaction.
FR-11	Display a budget alert when total spending exceeds ₹10,000.
FR-12	Save and load expense records using expenses.json.

4. Non-Functional Requirements

Requirement	How the project addresses it
Usability	Menu-driven options and direct console prompts.
Reliability	Validation functions reject invalid amount, category and date values.
Maintainability	Responsibilities are divided across focused Python modules.
Portability	Uses Python and standard-library functionality without a database server.
Data Persistence	Expense records are saved in a JSON file.
Error Handling	Invalid numeric input is caught and the user is prompted again.
Readability	Simple function names and small modules keep the implementation understandable.

5. System Architecture

The system follows a simple modular flow. The user interacts with **execution.py**, which loads saved expenses and routes menu selections to the appropriate functionality. Expense management handles record operations, reports perform calculations and budget checking, storage handles JSON persistence, validators enforce input rules and utils provides the title display helper.

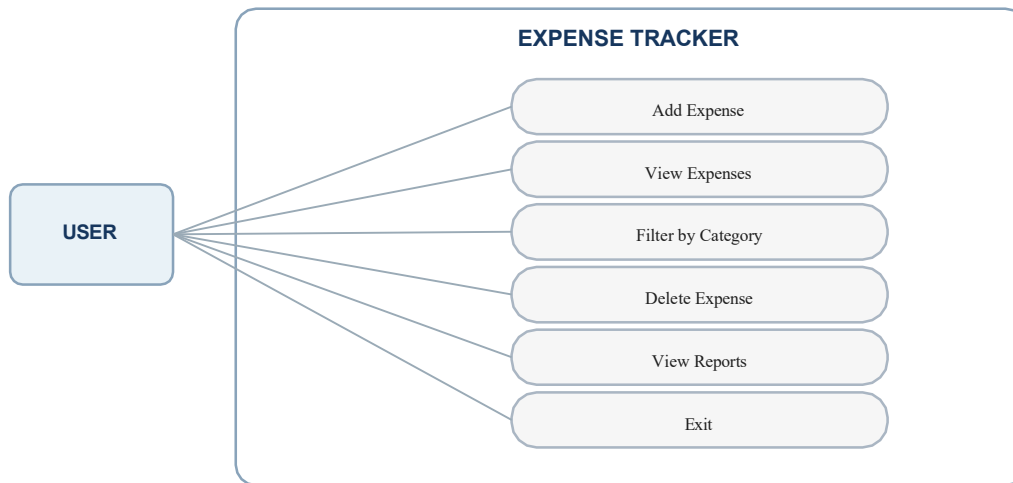


Architecture Components

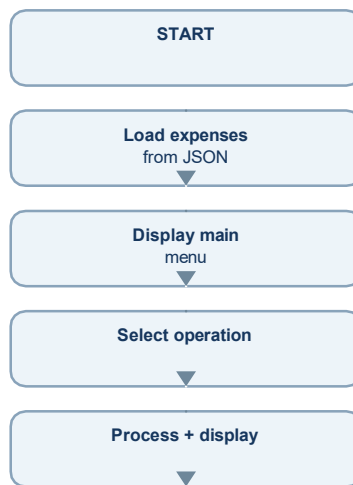
- **User Interface:** execution.py provides the console menu and input prompts.
- **Expense Management:** expense_manager.py contains add, view, filter and delete operations.
- **Reporting Layer:** reports.py calculates totals, category summaries, highest expense and budget alerts.
- **Storage Layer:** storage.py reads and writes expenses .json.
- **Validation Layer:** validators.py checks amount, category and date input.
- **Utility Layer:** utils.py provides the reusable title/display helper.

6. Design Diagrams

6.1 Use Case Diagram

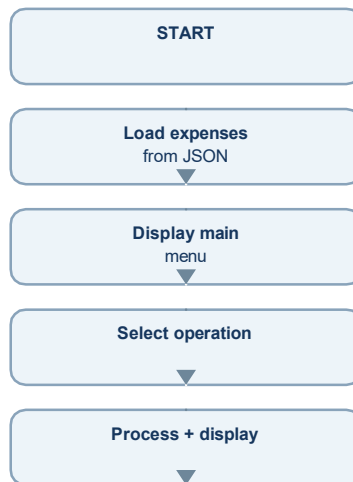


6.2 Workflow Diagram

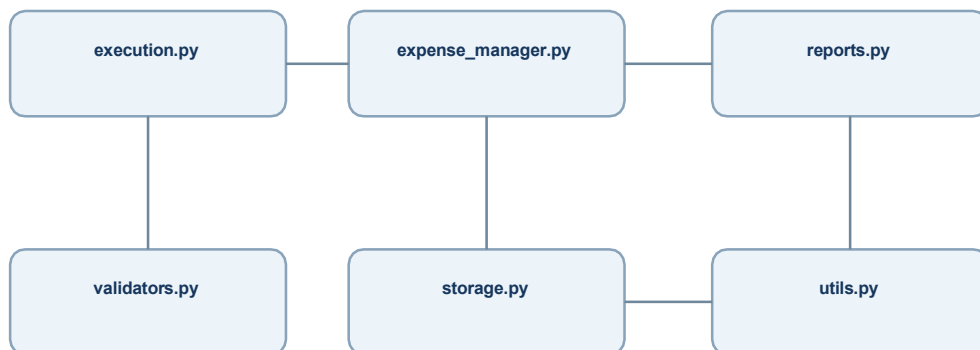


6.3 Sequence Diagram — Expense Operation

The simplified sequence below represents the normal flow of a user selecting an expense operation: the program loads stored data, displays the menu, accepts a choice, processes the requested operation and displays the result.



6.4 Component Diagram



Function-based modular implementation

The project uses Python functions rather than classes. A component/function diagram therefore describes the current implementation more accurately than a class diagram.

7. Data Storage Structure

The project does not use a relational database. Expense records are stored in a structured JSON file named **expenses.json**. The storage module exposes separate functions for writing the current expense list and loading it when the application starts.

EXPENSE RECORD	expenses.json
amount	numeric value
category	text value
date	YYYY-MM-DD string

Storage Flow

In-memory expense list → save_expenses() → expenses.json → load_expenses() → in-memory expense list

- JSON is human-readable and available through Python's standard library.
- The storage module uses a fixed file name, expenses.json.
- If the JSON file is missing, load_expenses() starts with an empty list.
- Persistence keeps records available between program runs.

8. Design Decisions and Rationale

Decision	Rationale
Use functions instead of classes	Keeps the project simple and demonstrates modular Python programming.
Use a list of dictionaries	Represents multiple expense records with clear named fields.
Use JSON storage	Provides persistence without external database setup.
Separate report functions	Each calculation has a focused responsibility.
Menu-driven navigation	Makes the console application easy to operate.
Validation functions	Keeps input rules separate from expense-management logic.
Category filtering	Allows users to inspect spending for one category.
Budget alert	Provides a simple threshold-based warning in reports.

Current Budget Threshold

Setting	Current implementation
Default budget limit	■ 10,000
Alert condition	Total expenses > ■ 10,000
Output	Warning message showing the exceeded limit and total spent.

9. Implementation Details

The implementation is organised into multiple Python source files. The main data flow is controlled by `execution.py`, while the remaining modules expose focused functions that are imported into the entry point.

9.1 Expense Management

- `add_expense(expenses)` accepts amount, category and date, validates them and appends a dictionary record.
- `view_expenses(expenses)` prints all records with numbered positions.
- `filter_expenses_by_category(expenses)` performs case-insensitive category matching.
- `delete_expense(expenses)` removes a selected record by its displayed number.

9.2 Reports

- `calculate_total(expenses)` returns the sum of all expense amounts.
- `category_summary(expenses)` accumulates spending for each category.
- `highest_expense(expenses)` returns the record with the greatest amount.
- `check_budget_alert(expenses, limit=10000)` prints a warning when the total exceeds the configured threshold.

9.3 Validation

- `validate_amount(amount)` accepts only values greater than zero.
- `validate_category(category)` requires non-empty text containing alphabetic characters.
- `validate_date(date)` uses `datetime.strptime` with the `YYYY-MM-DD` format.

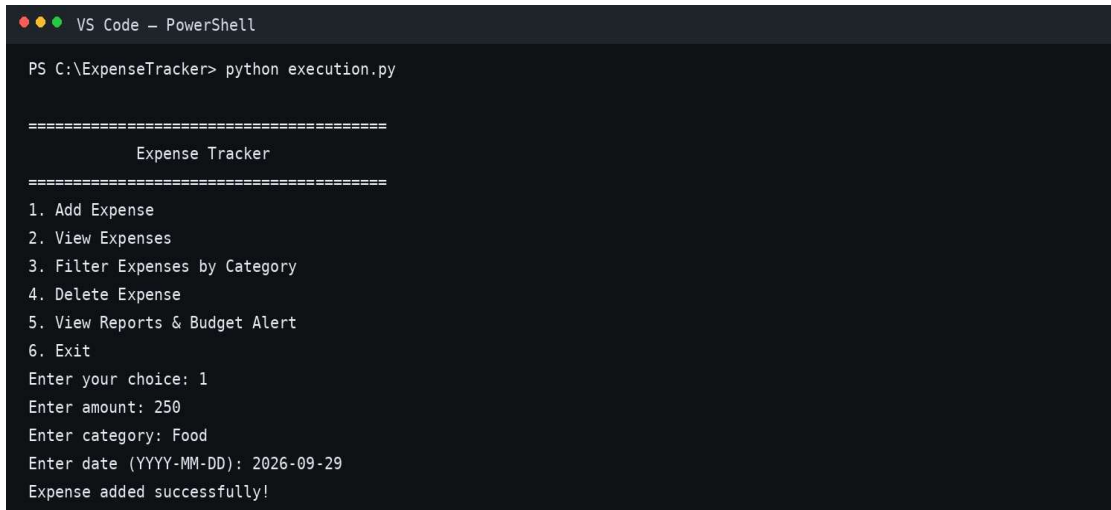
9.4 File Handling

- `save_expenses(expenses)` serializes the current list to `expenses.json`.
- `load_expenses()` loads the JSON file and returns an empty list if the file does not exist.

10. Screenshots / Results

The project is a console application, so its results appear as text screens rather than a graphical dashboard. The following screenshots document the actual Expense Tracker execution and validation flow.

10.1 Main Menu and Add Expense



```
VS Code - PowerShell

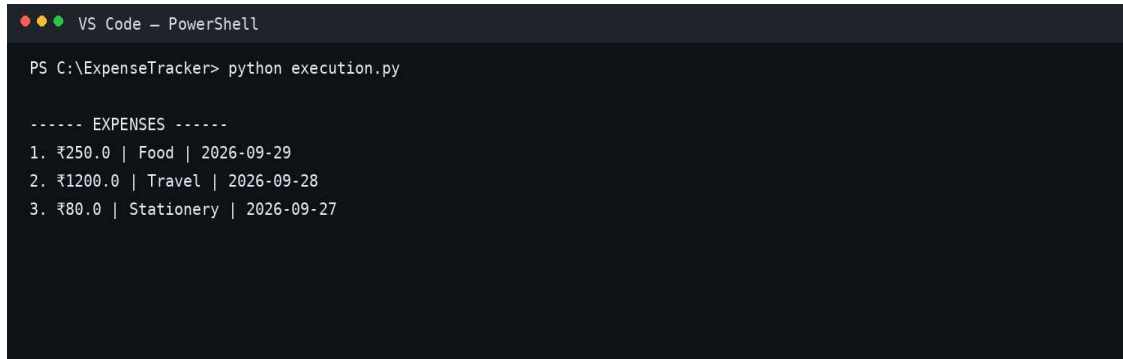
PS C:\ExpenseTracker> python execution.py

=====
                Expense Tracker
=====

1. Add Expense
2. View Expenses
3. Filter Expenses by Category
4. Delete Expense
5. View Reports & Budget Alert
6. Exit
Enter your choice: 1
Enter amount: 250
Enter category: Food
Enter date (YYYY-MM-DD): 2026-09-29
Expense added successfully!
```

Figure 10.1 — Actual project output showing the menu and successful expense entry.

10.2 View Expenses



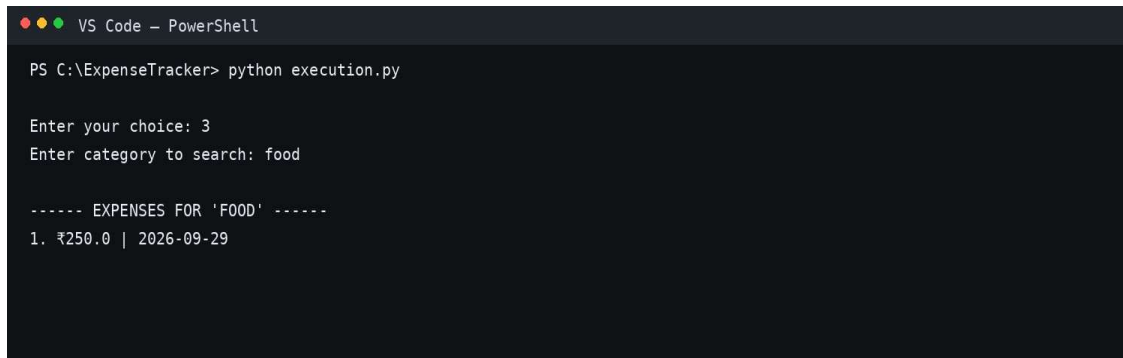
```
VS Code - PowerShell

PS C:\ExpenseTracker> python execution.py

----- EXPENSES -----
1. ₹250.0 | Food | 2026-09-29
2. ₹1200.0 | Travel | 2026-09-28
3. ₹80.0 | Stationery | 2026-09-27
```

Figure 10.2 — Actual project output displaying stored expenses.

10.3 Filter by Category



```
VS Code - PowerShell

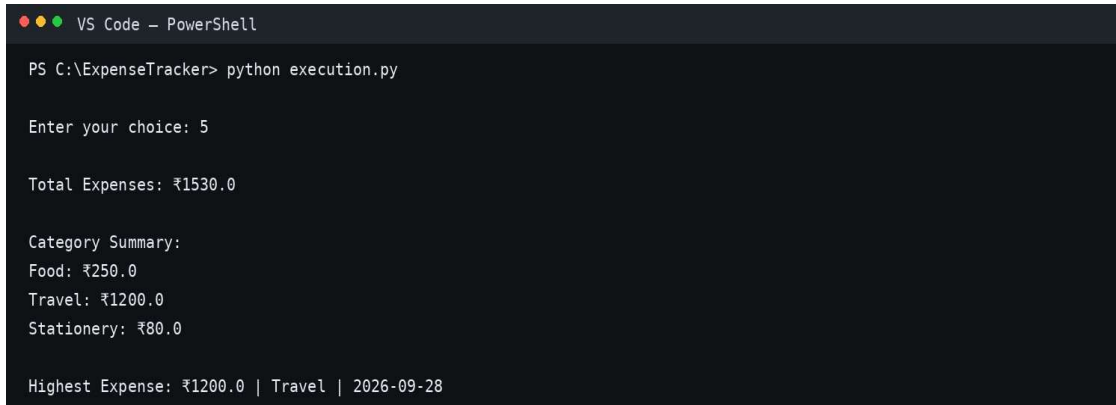
PS C:\ExpenseTracker> python execution.py

Enter your choice: 3
Enter category to search: food

----- EXPENSES FOR 'FOOD' -----
1. ₹250.0 | 2026-09-29
```

Figure 10.3 — Actual category-filtering output.

10.4 Reports and Budget Alert



```
VS Code - PowerShell

PS C:\ExpenseTracker> python execution.py

Enter your choice: 5

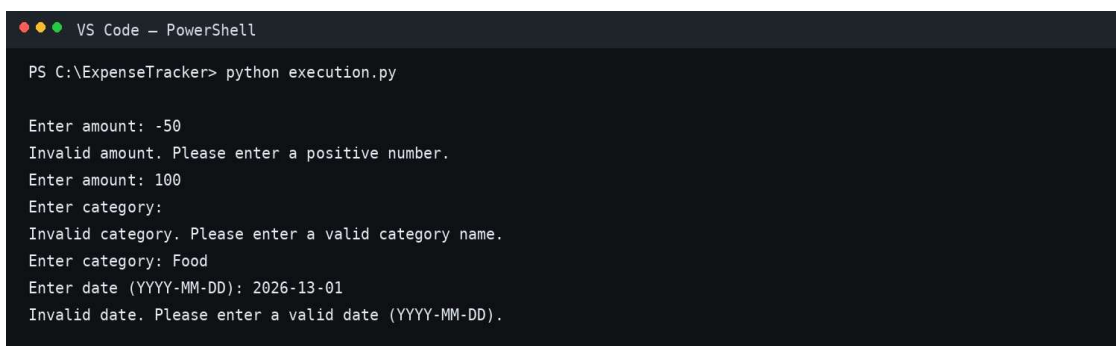
Total Expenses: ₹1530.0

Category Summary:
Food: ₹250.0
Travel: ₹1200.0
Stationery: ₹80.0

Highest Expense: ₹1200.0 | Travel | 2026-09-28
```

Figure 10.4 — Actual report output showing total spending, category summary and highest expense.

10.5 Input Validation



```
VS Code - PowerShell

PS C:\ExpenseTracker> python execution.py

Enter amount: -50
Invalid amount. Please enter a positive number.
Enter amount: 100
Enter category:
Invalid category. Please enter a valid category name.
Enter category: Food
Enter date (YYYY-MM-DD): 2026-13-01
Invalid date. Please enter a valid date (YYYY-MM-DD).
```

Figure 10.5 — Actual validation output for an invalid amount/category/date.

Result Interpretation

- The add flow accepts a valid amount, category and date.
- Stored records can be viewed directly from the console.
- Category filtering returns only matching records.
- Reports calculate total and category-wise spending and identify the highest transaction.
- Invalid inputs are rejected and the user is prompted again.

11. Testing Approach

Testing was planned around normal inputs, boundary values and invalid values. The repository includes a unittest test file under the test directory, and the project was also manually exercised through the console.

ID	Test Input / Condition	Expected Result
TC-01	Valid amount, category and date	Expense added and saved.
TC-02	Negative amount	Amount rejected.
TC-03	Zero amount	Amount rejected.
TC-04	Non-numeric amount	Input error handled and retry requested.
TC-05	Empty/invalid category	Category rejected.
TC-06	Invalid date	Date rejected.
TC-07	Existing category filter	Matching records displayed.
TC-08	Missing category	No matching records message.
TC-09	Valid delete number	Selected record removed.
TC-10	Invalid delete number	Invalid-number message.
TC-11	Report with records	Total, summary and highest expense shown.
TC-12	Missing expenses.json	Program starts with empty list.

Testing Note

The visible validation screenshot confirms that the application rejects a negative amount and invalid category/date input. The automated test file is also present in the repository under test/test_expenses.py. ■cite■turn0view0■

12. Challenges Faced

- Input validation: amount, category and date must follow meaningful rules.
- Data persistence: records must remain available after the application closes.
- Module integration: multiple Python files must work together through imports.
- Report calculations: totals, category summaries and the highest expense must remain consistent with stored records.
- Filtering: category matching should be convenient and case-insensitive.
- Delete handling: invalid numbers should not crash the application.
- Simple design: the project should remain understandable without unnecessary frameworks.

How the challenges were handled

- Validation logic was isolated in `validators.py`.
- JSON reading and writing were isolated in `storage.py`.
- Expense operations were separated into `expense_manager.py`.
- Report calculations and budget checking were placed in `reports.py`.
- `execution.py` acts as the entry point and coordinates the modules.
- Manual console tests and the `unittest` file were used to check behaviour.

13. Learnings & Key Takeaways

- Organised a Python application into reusable modules.
- Used lists and dictionaries for structured expense records.
- Applied loops, conditions, functions and exception handling.
- Implemented JSON file handling for persistent storage.
- Used validation functions to improve input reliability.
- Implemented category filtering.
- Converted raw records into total and category-wise reports.
- Implemented a highest-expense calculation.
- Added a budget-threshold alert.
- Used Git and GitHub for source-code version control.
- Understood the difference between in-memory data and persistent storage.

Key Takeaway

A useful application can be built from basic programming concepts when the problem is divided into clear, manageable modules. Separating validation, storage, reporting and record management makes the project easier to understand and extend.

14. Future Enhancements

Enhancement	Possible improvement
Graphical User Interface	Convert the console menus into a desktop GUI.
Web Application	Provide a browser-based expense dashboard.
Database Integration	Move from JSON to SQLite or another database.
Authentication	Add user accounts and separate personal expense data.
Charts	Show category distribution and monthly spending visually.
Export Reports	Generate CSV or PDF spending reports.
Budget Management	Allow users to configure budgets rather than using one default threshold.
Edit Expense	Allow users to modify an existing record.
Advanced Analytics	Add monthly trends and spending-pattern analysis.
Cloud Deployment	Host the application as a service.

Scope Note

These are proposed future enhancements only. They are not presented as completed features

Appendix — Project Feature Summary

- Language: Python 3.x
- Interface: Console / menu driven
- Storage: expenses.json
- Modules: execution.py, expense_manager.py, reports.py, storage.py, validators.py, utils.py
- Operations: Add, View, Filter, Delete
- Reports: Total Expenses, Category Summary, Highest Expense
- Budget Alert: ■ 10,000 default threshold
- Testing: unittest test file plus manual validation
- Student: Shashwat Shaurya — 26BCE10367

END OF REPORT